What is AWS Load Balancer?

www.linkedin.com/in/arun-kumar-akula

AWS Load Balancer is a service that automatically distributes incoming network traffic across multiple servers (like EC2 instances) to make applications more reliable, fast, and highly available.

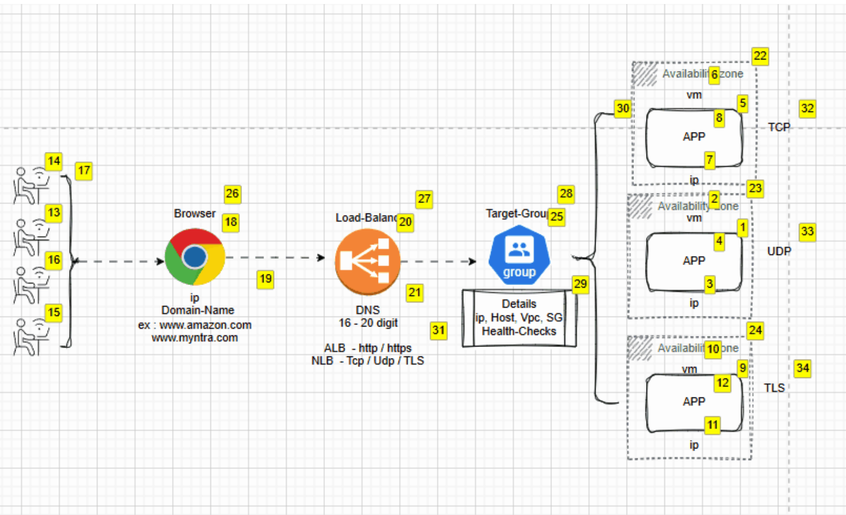
It helps to:

- Avoid overload on one server
- Redirect traffic to healthy servers
- Works with Auto Scaling to handle sudden traffic spikes.

Types of AWS Load Balancers:

1. **Application Load Balancer (ALB) :**
   - Works at Layer 7 (HTTP/HTTPS)
   - Best for web apps, path-based routing
2. **Network Load Balancer (NLB)**
   - Works at Layer 4 (TCP/UDP)
   - Best for high performance, low latency
3. **Gateway Load Balancer (GWLB)**
   - Used for third-party virtual appliances (firewalls, etc.)

Architecture of AWS Load Balancer :

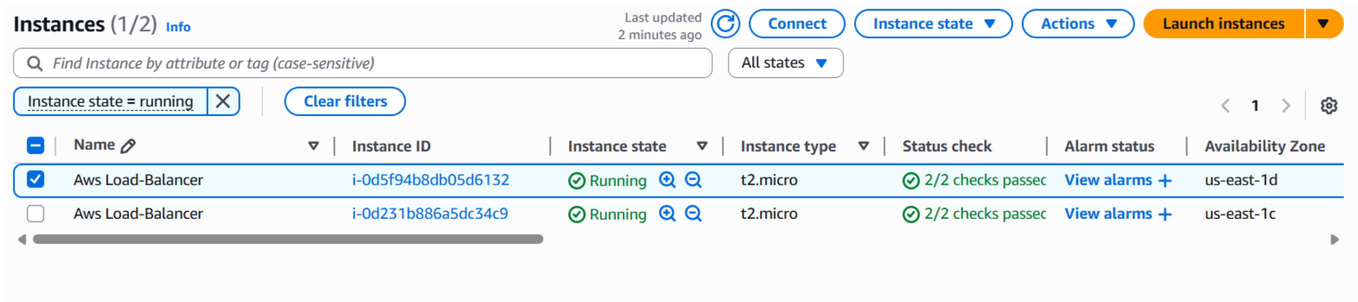


This architecture shows how users access a website via a browser using a domain name, which routes the request to an AWS Load Balancer (ALB/NLB) through DNS. The Load Balancer distributes traffic based on protocols (HTTP, HTTPS) to a Target Group that contains EC2 instance details like IP, VPC, security groups, and health checks. Finally, traffic is sent to healthy app servers running in different Availability Zones, ensuring high availability, fault tolerance, and smooth traffic handling

Let's walk through the practical steps to set up an AWS Load Balancer with EC2 servers

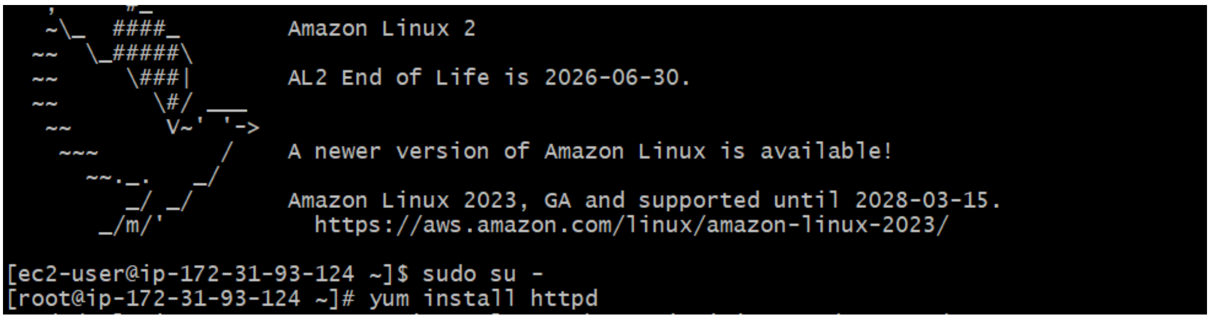
Step 1: Launch EC2 Instances (2 or more)

- **EC2 Dashboard** > **Launch Instance**
- **Amazon Linux 2** or **Ubuntu**
- Choose **t2.micro** (Free Tier)
- Add **HTTP (port 80) & SSH (port 22)** in security group
- Launch at least **2 instances** in **different Availability Zones.**



Step 2 : Connect to EC2 Instances and Download the Packages(Like httpd) in both servers.

Command to install httpd : sudo yum install httpd -y



Step 3 : Start the httpd server and enable the httpd

Command to start and enable the httpd :

- **Systemctl start httpd**

www.linkedin.com/in/arun-kumar-akula

- **Systemctl enable httpd**
- **Systemctl status httpd** (To the httpd server is active or not.!)

```
[root@ip-172-31-93-124 ~]# systemctl start httpd
[root@ip-172-31-93-124 ~]# systemctl enable httpd
Created symlink from /etc/systemd/system/multi-user.target.wants/httpd.service t
o /usr/lib/systemd/system/httpd.service.
[root@ip-172-31-93-124 ~]#
[root@ip-172-31-93-124 ~]#
[root@ip-172-31-93-124 ~]# vi /var/www/html/index.html
[root@ip-172-31-93-124 ~]#
```

Now, Create the web page using the vi command :

The httpd webserver default path is store web files **/var/www/html** , so we need to create a file in this path.

Command to Create a file : **vi index.html/var/www/html**

- Now **vi** will open a file and place HTML code ito it and save the file .

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Arun Kumar Akula - DevOps Engineer</title>
  <style>
    body {
      font-family: 'Segoe UI', sans-serif;
      background: linear-gradient(135deg, #ff416c, #9d50bb, #1e90ff);
      background-size: 400% 400%;
      animation: gradientBG 15s ease infinite;
      color: #fff;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
    }

    @keyframes gradientBG {
      0% { background-position: 0% 50%; }
      50% { background-position: 100% 50%; }
      100% { background-position: 0% 50%; }
    }

    .card {
      background: rgba(255, 255, 255, 0.1);
      border-radius: 20px;
      padding: 30px 40px;
      text-align: center;
      box-shadow: 0 8px 20px rgba(0, 0, 0, 0.3);
      backdrop-filter: blur(10px);
      max-width: 400px;
    }

    h1 {
      font-size: 34px;
      background: linear-gradient(to right, #ffe53b, #ff2525);
      -webkit-background-clip: text;
      -webkit-text-fill-color: transparent;
      margin-bottom: 12px;
    }

    h2 {
      font-size: 20px;
      color: #ffeb3b;
      margin-top: 0;
```

In Server-1 vi file, I mentioned the code as **'Arun Kumar Akula'**, and in Server-2 vi file, I used **'Arun Kumar'**. This was done to observe traffic distribution when we refreshed the page."

AWS Placement Groups?

Placement Groups in AWS are used to control how EC2 instances are placed within the AWS infrastructure to improve performance, availability.

Why Attach Placement Groups to EC2 Instances?

You use Placement Groups to get better performance, reliability, or fault isolation depending on your app needs.

Types of Placement Groups :

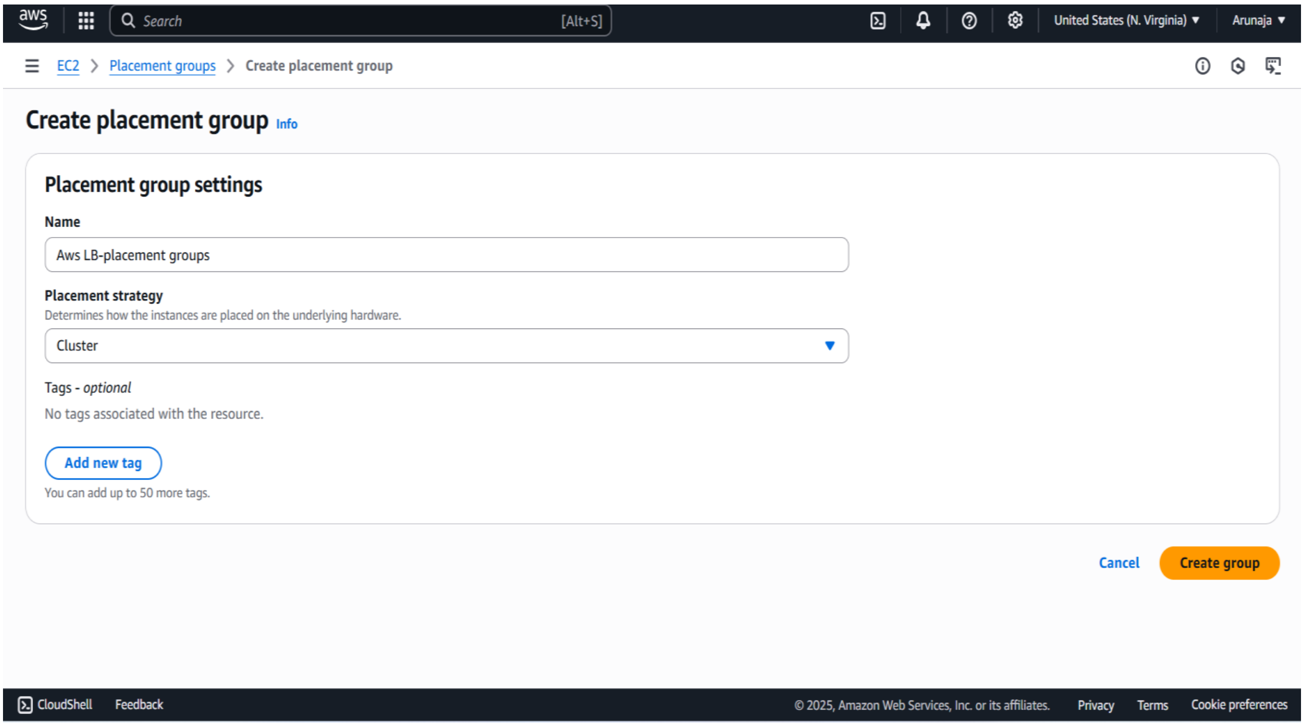
1. Cluster: Places instances close together for high speed and low latency.

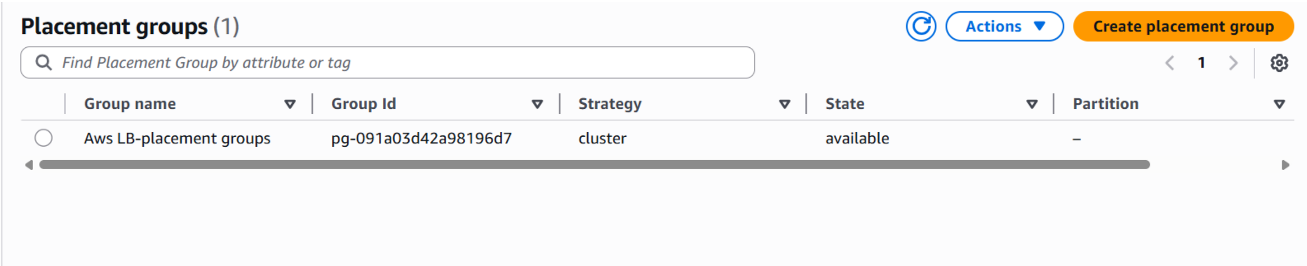
2. Spread: Places instances on separate hardware to reduce failure risk.

3. Partition: Groups instances across separate racks for fault tolerance.

Step 4 : Create the placement groups

- Go to EC2 Dashboard in the AWS Console.
- In the left menu, click Placement Groups.
- Click Create placement group**.**
- Give it a name (e.g., my-placement-group).
- Choose the strategy: **Cluster, Spread, or Partition**
- Click Create placement group.





What is a Target Group in AWS :

A Target Group is a collection of servers (like EC2s) that a Load Balancer sends traffic to, and it also stores all the details related to those servers**, like IPs, ports, health check status,** and **protocols.**

Why does a Load Balancer attach to a Target Group:

The Load Balancer doesn't talk to EC2s directly it talks to **Target Groups**, which manage the actual servers.

Step 5 : Create a Target Group

- Go to the EC2 Dashboard in AWS Console.
- On the left menu, click Target Groups
- Click Create target group

Fill the Required Details:

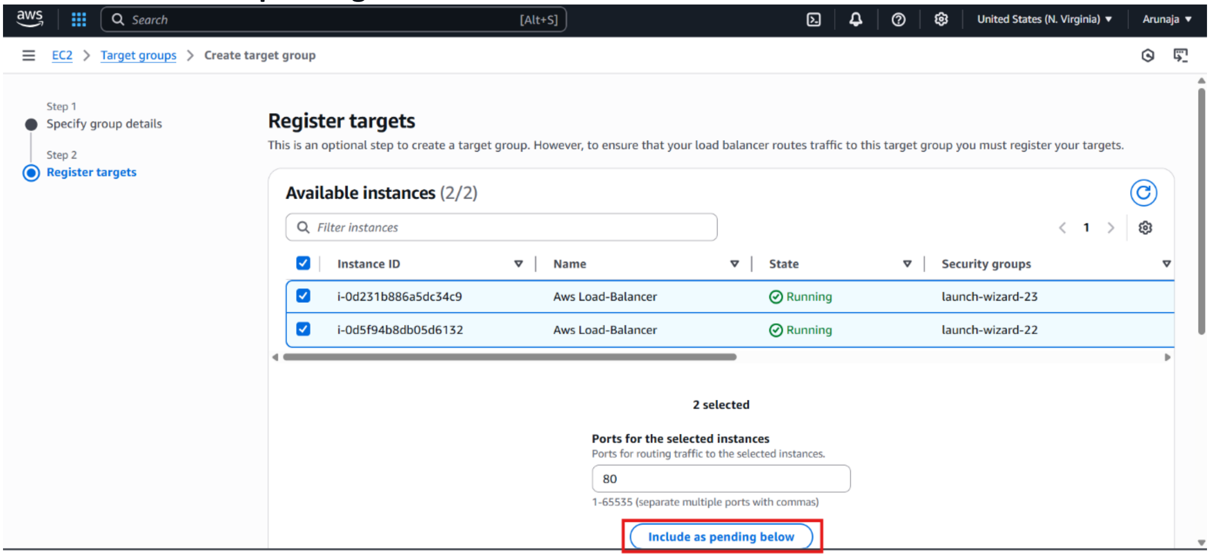
- Target type: Choose Instances (for EC2)
- Target group name: Example — my-target-group
- Protocol: HTTP
- Port: 80
- VPC: Choose the same VPC as your EC2 instances
- Protocol version: HTTP1

Configure Health Checks:

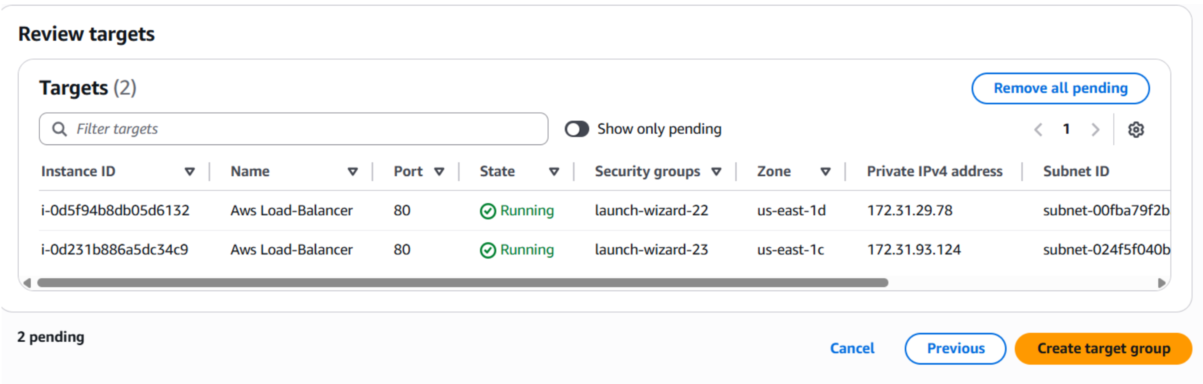
- Protocol: HTTP
- Path: / (default)

Click Next to register targets www.linkedin.com/in/arun-kumar-akula

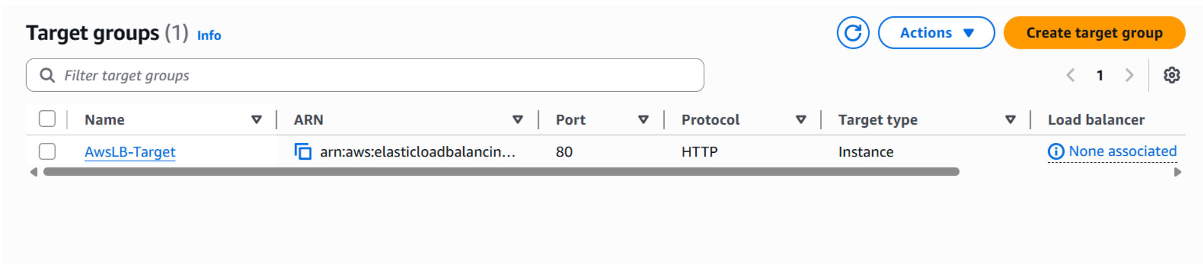
- Select the EC2 instances you want to include
- Click **Include as pending**



- Click **Create target group**



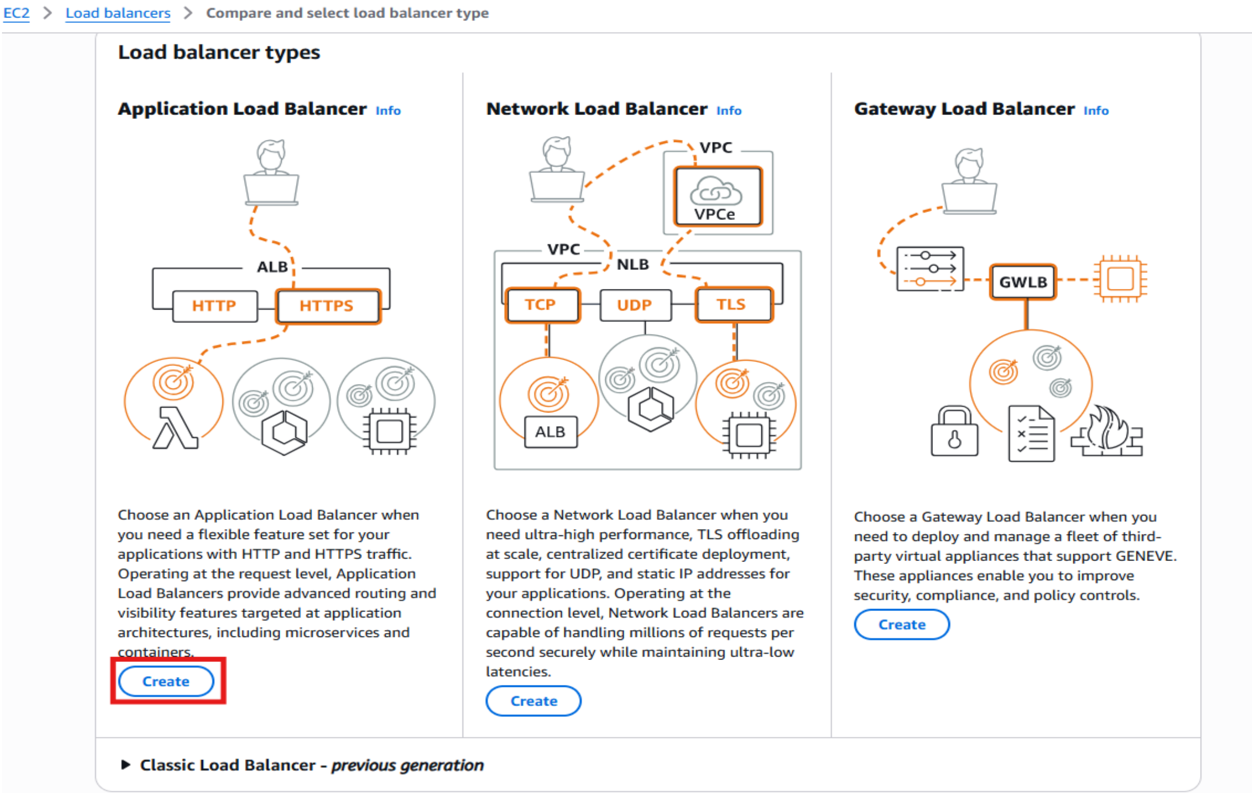
- Now we can see the created Target group in the Target groups Dashboard.



Step 6 : Create an Application Load Balancer (ALB)

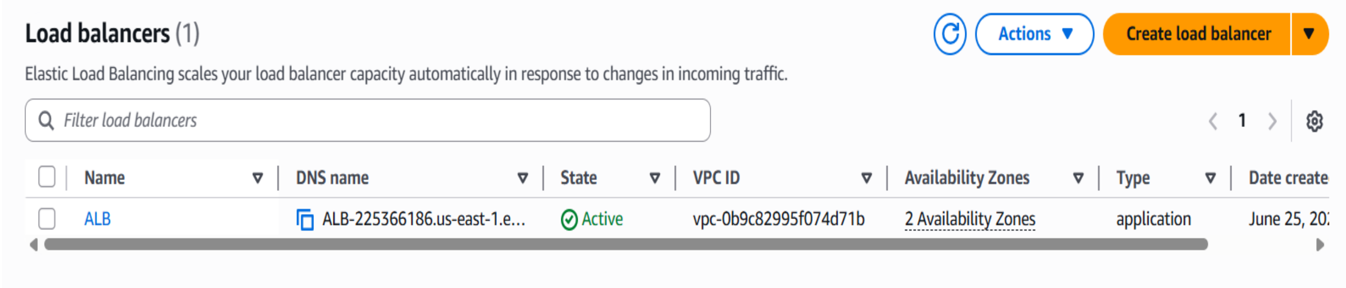
- Go to the EC2 Dashboard
- In the left menu, click Load Balancers
- Click Create Load Balancer

- Choose Application Load Balancer

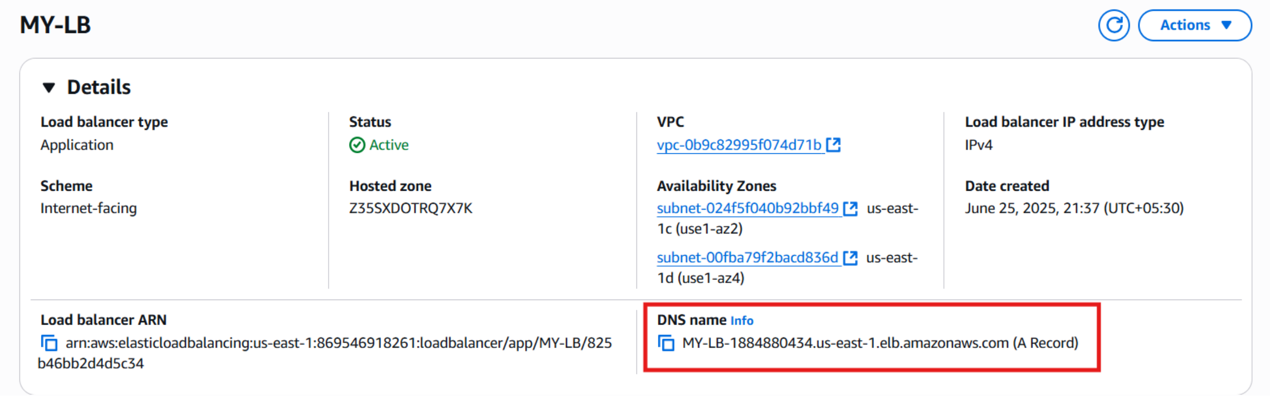


Fill the Basic Configuration :

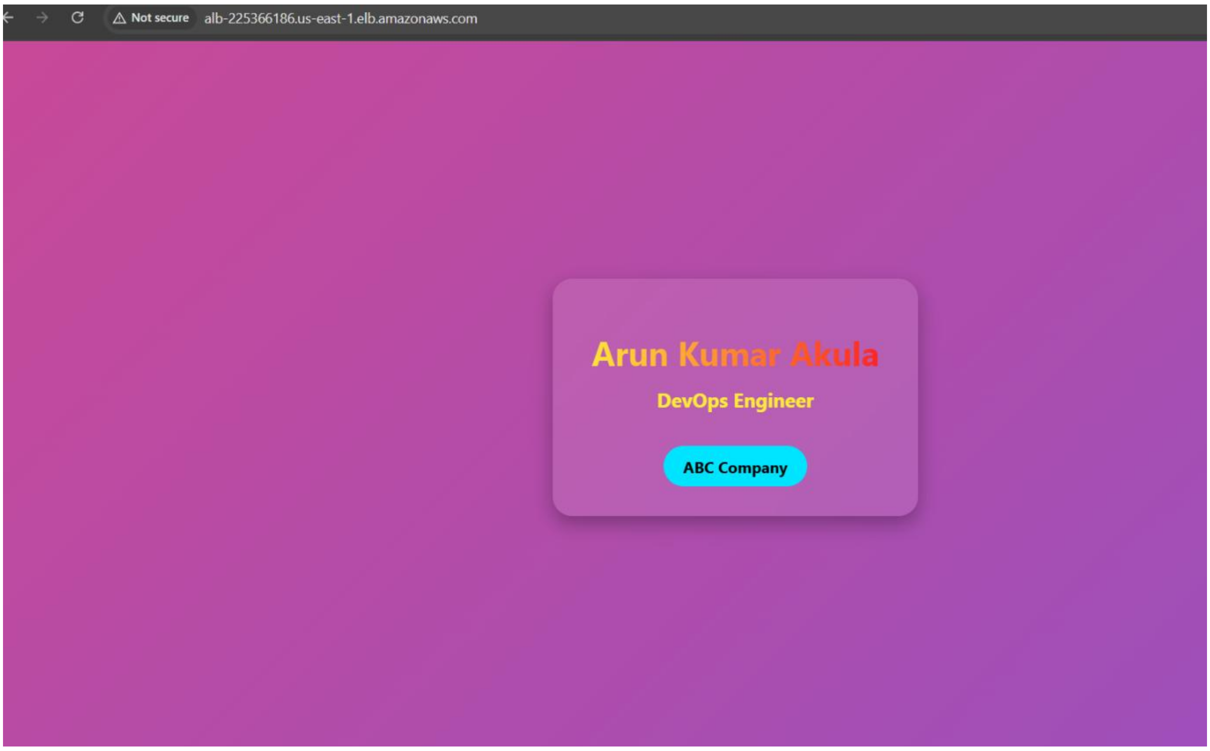
- Name: e.g., my-alb
- Scheme: Internet-facing (for public access)
- IP address type: IPv4
- VPC **:** Choose the same VPC as your EC2 instances
- Availability Zones and subnets **:** Select at least 2 Availability Zones — the same ones where your EC2 instances are running
- Security groups : Choose the same Security Group that your EC2 instances are using.
- Listeners: Keep HTTP on port 80
- Click **Create Load Balancer** (After a few minutes, your ALB will be ready.)



- Go to Load Balancer > Description Tab
- Copy the DNS name



- Paste in your browser to test it.!



Load Balancer Output :

- **Server 1 shows**: Arun Kumar Akula
- **Server 2 shows**: Arun Kumar

When you open the Load Balancer DNS and refresh the page, the output changes between both this shows traffic is going to both servers correctly.

www.linkedin.com/in/arun-kumar-akula